When Abstraction Becomes Indirection:

Tokens-to-trace, measured on four stacks

Vasili Gavrilov

August 2026

Abstract

We examine whether programs written with AI coding agents still need the framework layers, language abstractions and syntactic sugar invented for human coders. An agent must read everything that decides what code will do before changing it safely. We measure that reading as tokens-to-trace: for one entry point, the token count of the smallest set of source regions that determines its behavior, each region classified by whether its text lives in the project, in library source, or only in the version's documentation. Prior work counts the tokens agents spend; ours counts the deciding text, and where it lives, before any run. We measured one list path in four real systems, and the ratios are the result: idiomatic C++ (taskwarrior) needs 2.8 times the plain C path's text counted the same way (257 against 90 thousand tokens), a gap that is mostly feature scope: matched for behavior in seven small pairs, the idiom costs about 15% more text, spread over up to eight files against C's one; Spring Boot 4 needs at least 51 to 124 thousand tokens of framework text outside the project where plain Java with JDBC needs none (8.2 against 7.6 thousand inside it), and part of the framework's behavior is in no repository. In 280 paired agent runs the layered side cost more mean turns in all seven pairs, about 20% on average, all but two runs passing their hidden tests; only the three largest costs stand above noise, and a control pair shows the cost is the file scatter, not the dispatch. One cost we did not measure, the implicit context a reader must bring: for C it is the K&R book, 272 pages; for C++ the Stroustrup, 1,368; for a framework, versioned documentation with no fixed size. We conclude that one line of style bias in the prompt gives an agent the whole searchable truth of a path for a few thousand tokens, where the layered styles fill its context or require text no repository supplies.

1 The question and the measure

A class hides a representation and an interface hides an implementation. A framework hides something larger: control flow. Each is a remedy for a limit of human working memory. An agent is not under that limit and is under others: the context it carries and the relationships it must resolve before acting. A hidden implementation is a relationship to resolve, which abstraction adds rather than removes.

The machine-cost objection to abstraction was answered decades ago by compiler work and the zero-overhead principle, and nothing in this paper rests on machine cost (the wall-time numbers are in the five-language bench in `ais` [9]). The reader's cost was never priced, and the reader has changed since.

$\mathrm{tokens\text{-}to\text{-}trace}(e)$ is the token count of the minimal set of source regions that fully determines the behavior of one entry point e, wire to storage and back: routing, authentication, authorization, handler logic, query, schema, serialization, error mapping. A region is in the set iff removing it leaves some behavior of e undetermined. The platform floor (JDK, libc, servlet container, database engine, vendored third-party crypto) is excluded identically everywhere; anything the application configures, or that varies by library version, is included. Each region is classified by residence:

1. **repository**: read directly from the project's own source;
2. **library source**: text outside the repository that decides behavior before the program runs;
3. **convention**: fixed by defaults of the exact version in use, recoverable from its documentation or from a runtime observation, not from any source file.

In the agent's own terms the closure is the needed context: what must be in front of it before it can decide that a change is safe. Template specialization and virtual-override resolution are classified

by the residence of the deciding code. Tokens are o200k, the encoding of the current OpenAI models, counted with tiktoken, OpenAI's open-source tokenizer library [24]: a public, model-neutral count that approximates what a commercial agent is billed (the pilot's own billing used the model vendor's tokenizer, which differs); the region manifests, with per-row lines, characters and tokens, are `closures.py` and `closures.json` beside this paper. The measure is a count: anyone can recompute it from the manifest, so two systems compare by number, and a disagreement is about a manifest row, not about an interpretation.

2 Four closures

One read path per system, each doing the same kind of work so the numbers compare: an authenticated or filtered list over a local store, returning rows.

ais, plain C. An associative index [9], 10,597 code lines, written to an explicit style guide [8] in the school of K&R and Robbins [1, 2] (MISRA-inspired; compliance with MISRA C:2012 [4] is not claimed or checked). The non-interactive recall path (`ais KEY1 KEY2`: AND-intersection over posting lists, record printing, document-blob branch) closes in **489 lines across 6 files, 5,750 tokens** with the storage engine as floor; for the Java pair below, MySQL is that floor. The interactive secret-reveal branch, taken only at a tty, adds 3,634 tokens including the contract of the crypto wrapper (the vendored Monocypher beneath it counts as floor, a rule added in this revision with Monocypher its only instance). With the hand-written engine added (posting merge, key folding, ledger scan, tombstones, crash recovery, headers as contracts), the determination of the non-interactive path from `argv` to the bytes on disk is **1,345 lines across 16 files, 14,444 tokens**: 7% of one window. The rows overlap by one 207-token comment (5,750 + 8,901 - 207 = 14,444); with the reveal branch the total is 18,078, and then nothing but Monocypher is floor. Whole files over the same 16: 90,151. The callbacks on the path are function pointers passed at the call site; the ops-struct vtable idiom of sqlite or nginx would resolve less cheaply, so this row measures one explicit style, and an abstract-C row (nginx's request path) is future work.

Monocypher 4.0.3, third-party plain C. To bound how much of the plain rows is the author rather than the style, the same rule was applied to a library nobody here wrote: Monocypher [13], the cipher ais vendors, widely audited, and the paper's own example of the algorithmic style, functions over buffers. Its whole AEAD encrypt path (`crypto_aead_lock`: the XChaCha20 keystream, the Poly1305 tag, the wipe) closes in **715 lines across 2 files, 7,560 tokens**, beside System A's 7,566 and ais's 5,750. The posture differs by design, a cipher rather than a list, and that is the point: at the algorithmic extreme the plain style's closure is the same few thousand tokens whoever writes it.

taskwarrior 2.6.2, idiomatic C++. A widely used open-source command-line task manager, in development since 2006 [14], whose `task list` does the same kind of work: filter records from a local plain-text store, print a table. Version 2.6.2 is the last all-C++ release (3.x moved the core to Rust), which is the reason it is the subject. Its two dispatch registries, commands and columns, are filled once at startup from literal constructor calls, so they resolve statically at their registration sites, one hop more than a call by name; an earlier draft of this paper called that granularity undefinable, which was wrong, and the retraction corrects the claim, not the number, since the measured set (`CmdCustom`, the 13 columns of the default report, their five base classes) was already the resolved one. The closure at whole-file granularity is **99 files, 33,706 lines, 256,664 tokens**, with 43 `virtual` declarations and 26 `overrides` on the path; with the store (its TDB2 class and file layer) as floor, 239,699. Compared at the same granularity, whole files on both sides, the C closure is **90k against the C++ 257k, a factor of 2.8 in tokens and 6 in files (16 against 99)**; regions inside files were not cut out on either side of that comparison, so both numbers are upper bounds of the same kind. A single system does not establish the idiom's cost: the whole of leveldb, a well-factored idiomatic C++ codebase, is 136k o200k tokens, so its read-path closure would land far below taskwarrior's; n here is 1, and the feature scopes of `task list` and `ais` recall differ (urgency, color rules, virtual tags on one side).

System A, plain Java with JDBC. A commercial web application in production, 39,677 code lines, reported anonymously. The endpoint (token authentication, role gate, per-object ACL, a 4-table JOIN,

streamed JSON, central error mapping) closes in **813 lines across 17 files, 7,566 tokens**: 3.8% of a 200k context window. The schema enters as the frozen base DDL plus the forward-only migrations that alter the seven tables the query and its ACL clause touch, because that sum is the live schema: the base is never rebuilt and every change moves forward, so the data survives every revision. Reading every touched file whole instead of the exact regions costs roughly 100k, still one window, and every callee on the path is named literally at its call site, so each hop is findable by one search. Categories 2 and 3 measure zero at the stated floor.

Side note: the same parser, written plain. The filter machinery on the measured path is a small language: `Lexer`, `Variant` with its 30 operator overloads, and `Eval`, 4,934 lines across 6 files. The prompt that asks for its plain form is one line: "plain C, ais-style, MISRA-inspired, the way ciphers and codecs are written: structs and functions, fixed bounds." What that form looks like:

```
struct token  { enum { T_NUM, T_STR, T_DATE, T_OP, T_END } kind;
                const char *s; size_t n; };

struct value  { enum { V_NUM, V_STR, V_DATE, V_BOOL } kind;
                union { double num;
                        struct { const char *s; size_t n; } str;
                        long   date;
                        int    flag; } u; };

struct cursor { const char *p; };

static int next_token(struct cursor *c, struct token *t);
        /* one switch over *c->p; no allocation, the token
           points into the input */
static int eval(struct cursor *c, const struct task *t,
                struct value *out);
        /* recursive descent; each grammar rule one function,
           each operator one case of one switch */
```

The 30 overloads become cases of the switch in `eval`; the three classes become two structs and a cursor; the token and the value live on the stack and point into the input, so the memory cost is the input plus a few dozen bytes. This is a sketch, not a measured artifact: the fair experiment would be to write it, pass taskwarrior's own tests, and count it, and the pairs of Table 3 already price the constructs it removes. It is shown because the claim "the plain form exists and is small" is otherwise abstract.

Spring Boot 4.1.0, framework Java. `raeperd/realworld-springboot-java` v2.1.1 [15], an implementation of RealWorld, the community specification of one Medium-style blogging application that many stacks implement identically so the implementations can be compared; this one is conventional, well-regarded Spring (Framework 7.0.8, Security 7.1.0, Data JPA 4.1.0, Hibernate 7.4.1, Jackson 3.1.4, resolved from the Boot BOM). Endpoint `GET /articles?author=...`: authenticated, paginated, viewer-dependent flag, five entities. Category 1: **27 files, 1,267 lines, 8,175 tokens**, 8% more than System A's whole closure for a domain without study design, supervision or reporting behind it. Category 2, the 27 framework classes on the dispatch path, measured from the published sources jars of the exact versions:

Module	Classes	Lines	o200k
spring-webmvc	9	6,310	52,178
spring-web	4	1,310	10,758
spring-security (web, config, core)	6	3,348	29,498
spring-data-commons	5	1,959	14,572
spring-data-jpa	3	2,282	16,788
Total	27	15,209	123,794

Table 1: Category 2: framework source whose text decides the endpoint's behavior, whole classes on the dispatch path. The class list is one measurer's trace of the path; a blind second rater's trace of the same path measures 373k tokens (the threats section reports the agreement), so this table is the conservative trace. Beneath both, uncounted in any closure, hibernate-core measures 6.7M tokens of source (one tiktoken pass, recorded in `closures.json`). Drawing the floor above MVC dispatch and the filter chain, on the argument that they are as stable as a servlet container, leaves 51,342: the interpreters of this application's own text (derived-query parsing, `Pageable` resolution, the repository proxy, and the `HttpSecurity` DSL that turns its security configuration into a filter chain).

Category 2 says where the deciding text lives, not what an agent reads: in practice agents finish Spring tasks without opening framework source at all, because the conventions sit in the training weights (that is practitioner report, not a measurement of this paper), and this moves the cost into category 3, where the corpus is a superposition of versions, rather than removing it. The failures practitioners report agents making on Spring, again report rather than measurement, are category-3-shaped: the `javax`-to-`jakarta` migration, the removal of `WebSecurityConfigurerAdapter` in Security 6, `@Transactional` self-invocation not proxying, lazy loading outside a session, dialect drift between H2 and production. The plain stack ate the `jakarta` rename too; only the surface differs: one import namespace with identical semantics against a DSL whose semantics moved. Category 3 for this endpoint, enumerated: the derived-query grammar (`findAllByAuthorProfileUserName` becomes a join tree by property-path rules; the SQL is in no repository file and is recovered by turning on Hibernate's statement logging for one run, or reconstructed from the rules); mapping precedence among four same-URL methods discriminated by `params`, an idiosyncrasy of this application (the common idiom, one method with optional parameters, removes that machinery from the path); `Pageable` defaults; which beans auto-configuration creates; filter-chain order from the `HttpSecurity` DSL; JPA fetch and flush defaults; serialization defaults; H2's PostgreSQL emulation. The Jackson 3 package rename (`tools.jackson`) and the Boot 4 module split are the version-incoherence instances observed during measurement.

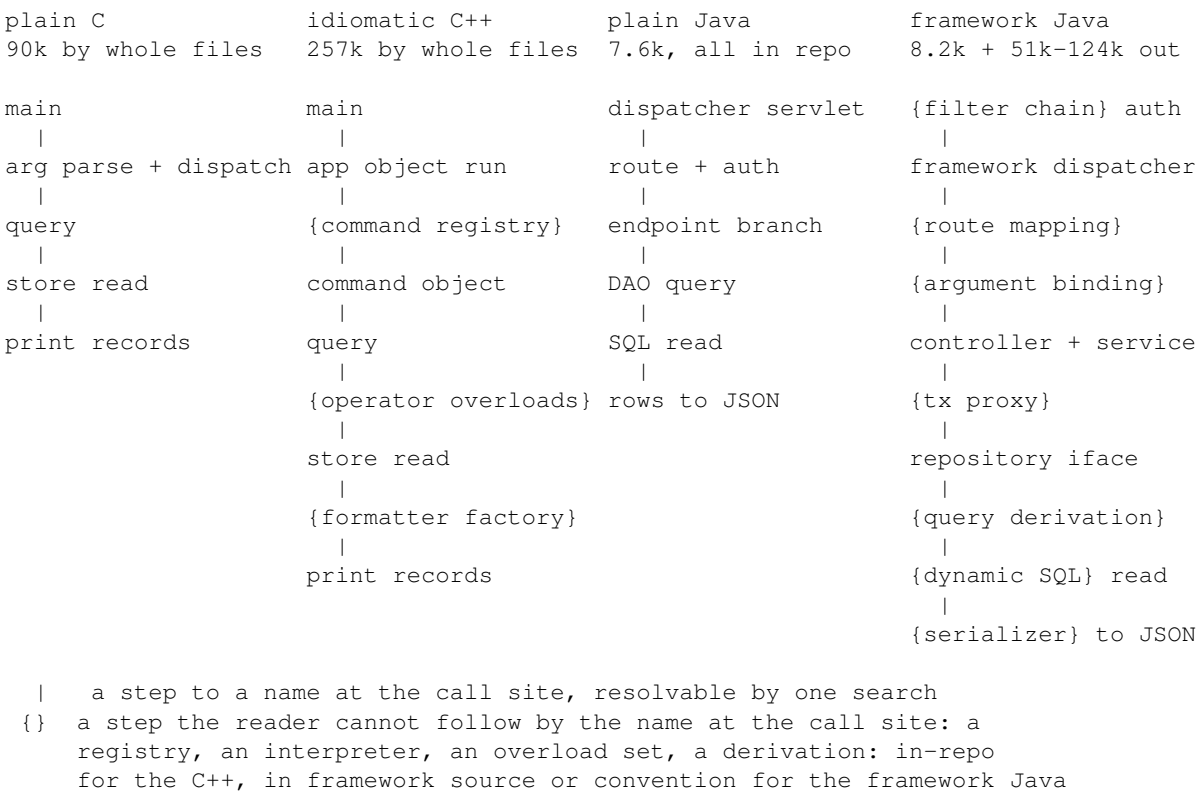

Figure 1: The path a reading agent resolves through the four measured endpoints, the plain and layered member of each language pair side by side. Every step is a role of the style, not a name from the project, so the columns contrast the styles: each layered column is its plain sibling's path with the style's resolution steps inserted. In the measured C++ the registry is the application object's command map, the command object one class per command, the overload set the value type's 30 operators, the factory the output-column registry; in the measured framework the dispatcher is Spring's `DispatcherServlet`, the route mapping its annotation conditions, the argument binding its parameter and paging resolvers, the query derivation `PartTree` over the repository method name, the serializer Jackson. The columns are matched in the kind of work, an authenticated or filtered list over a local store, not in feature scope; on the framework path the SQL and the serialization defaults have no file of their own in the repository, and the closures under the headings differ by the factors of Table 2. The inserted steps are not drawing choices: the pairs experiment prices them (Table 3), and its control shows the one-class-per-file layout, not the dispatch construct, carries most of the largest cost.

System	Style	Cat. 1	Cat. 2	Cat. 3
ais, engine as floor	plain C	5,750	0	0
ais, engine included	plain C	14,444	0	0
taskwarrior, whole-file*	idiomatic C++	256,664	0	0
System A	plain Java + JDBC	7,566	0	0
Monocypher AEAD (third party)	plain C	7,560	0	0
Spring endpoint	framework Java	8,175	123,794†	>0§

Table 2: Tokens-to-trace by residence, one read path per system, o200k. Category 1 is text in the system's own repository, category 2 text in library or framework source outside it, category 3 behavior fixed by the version's conventions, with no source text anywhere. A zero means every region that decides the path's behavior is in the repository: the first four rows use libraries only below the floor, and their dispatch machinery, taskwarrior's included, is their own code, while the framework row's deciding text sits partly in the Spring jars and partly in version convention. *Upper bound; measured the same way, whole files, the C figure is 90k, factor 2.8. †51k under the highest defensible floor (Table 1). §Present and enumerated in the text; not counted, since its residence is documentation and runtime observation, not source.

Information hiding lets a client not read the implementation: a well-typed client cannot observe it, so for a client-side task the interface and its contract are the whole read, and the closure measured here is an upper bound the reader rarely pays. The runs agree: the median run read three or four files, not the closure. The closure matters when that protection stops working: a defect or a semantics change sits behind the boundary and the reader must go in. The cross-module tasks of the pairs experiment simulate that case, and maintenance eventually produces it. The residence claim is untouched by the objection either way: a convention that is in no repository cannot be read at any granularity. The next measurement this distinction asks for is task-conditioned reading beside worst-case closure, on the same systems.

Each kind of step has a concrete cost to the reader, set by the resolving procedure, and the experiment's numbers are consistent with it at the ends. A name at the call site is one search: the callee's text is one grep away, which is why the plain runs are a few reads and an edit. A registry adds one search before the first one can land, finding the registration site; measured, its pair's cost is mostly the walk across the idiom's files, and what is left for the dispatch itself does not resolve against noise. A template call resolves in one search once the argument types are known, $+1.70$ measured. An overloaded operator leaves a greppable token but not a unique name: the reader must establish the operand's type before the grep lands, a cost too small to resolve in these pairs, and argument-dependent lookup, which can put the deciding candidate in any associated namespace under a rule written in no source file, is the harder form of the same cost, not exercised by these pairs. Inheritance and RAII stay under two turns and inside the batch noise; RAII's real reading obligation is not finding the destructor, which sits beside its type, but knowing where destructors run, an implicit control flow on every exit path, and the smallness of its measured cost says agents carry that obligation easily. The reader also holds an oracle the procedure omits: the compiler. A build confirms a resolution hypothesis in one turn, the runs used it constantly, and it discounts every static-resolution step above, though not the derived query, which no compiler in the repository can check. A derived query name resolves through a grammar that is in no repository file, so no number of searches inside the repository ends the read: the reader recalls the grammar, reads the derivation source priced in category 2, or observes the runtime, which is the framework row's residence problem. The recorded runs are small enough to sweep, and their traces show both readings: across all 280 the tool counts are 911 shell calls, 810 file reads, 412 edits, and 6 writes, the harness's search tool was never called, and 154 of the shell calls carry a grep, about two thirds of them searching the source and the rest filtering file listings, so the agent enumerates and reads at this scale and still searches when a name is worth finding. File count converts to reads and reads to turns, which is the mechanical form of the control result; what stays untested is the price of grep-per-hop reading in a repository too large to sweep, where the search would be the only mechanism left. The procedure was written after the first results; recorded in advance was only the clause that the explicit side's advantage grows with the number of plausible dispatch targets per call site.

The plain closures fit in a context window beside the work. The framework's category 3 is text only in reference manuals that are not version-pinned in any repository, and the training corpus superposes their versions: a coherence problem, which window growth does not remove, while it does absorb the C++ row's volume (a 1M-token window holds 257k beside the work). That makes the framework claim the stronger of the two, and the C++ claim a matter of volume and of counting granularity. The paper's own taxonomy separates them, and the hypothesis in the abstract rests on the framework row.

3 What an agent pays: two bounded experiments

Mechanism. Sixty maintenance runs, in which the agent was told nothing of the study (Claude Code headless, `claude-sonnet-5`, four tasks that name behavior and never a file or a function, five languages, hidden tests). The source under maintenance was at most 1.1% of any run's context (the largest fixture tree, 1,805 o200k tokens, against the smallest run context), while the median run re-sent 25,603 tokens every exchange, most of it the fixed preamble the harness sends each time plus the accumulated conversation, nearly all of it served from prompt cache at a fraction of full price, so a whole run cost $0.11. The pilot establishes the shape of the cost: it is paid per exchange and dominated by what is carried, and the five representations did not separate beyond run-to-run spread, though no formal test was run at twelve runs per language. It cannot establish any claim about scale, because every run sat far below the window. The cost of a closure beyond the window, retrieval hops and the working set after retrieval, remains for the experiments named in Section 6.

Repeated sites. The don't-repeat-yourself rule, DRY, exists because a human asked to change N call sites misses some. Twenty-five runs asked for one behavioral change that must land at every site, at 1, 3, 10 and 30 repetitions plus a factored control, task text naming no site. Coverage was 30 of 30 in every run that had 30 sites, and 100% in every condition. This is a ceiling result: at five runs per condition, a per-run miss probability up to ≈45% is not excluded (the one-sided 95% upper bound at 0 of 5). The sites were exact copies one screen apart in one file, and the task text stated universal scope. The run records sharpen the ceiling further: the 30-site runs finish in five or six tool calls with zero per-site edits, one replace-all pass over identical text, so coverage here is a property of the edit tooling as much as of the model, and site count is irrelevant by construction. Only one thing stands: the easiest form of the hazard cannot occur this way; no single pass ends the field form, near-miss duplicates scattered across files (14 to 53% normalized against 3.9 to 21.8% exact in the public corpora), and it is untested.

4 The shape of the difference, before any agent runs

Public corpora, 1.74M code lines measured with one script: C carries 28.4 functions per thousand lines, C++ 60.0, Rust 72.5, and the median C++ function has cyclomatic complexity 1: no branch, which is consistent with a delegation-heavy style though a branchless function can also compute. Complexity concentrates in C (a maximum of 1,330 in sqlite) and spreads thin in Rust (25 in tokio). Genre is confounded with language here: the C side is servers and databases, half the C++ side is header-only template libraries. One measured contamination is corrected in the shipped data: spdlog vendors a copy of fmt under `bundled/`, and vendored trees are excluded from the corpus counts. The corpus establishes one thing only: the indirection whose cost is in question is real, large, and ordered from C through C++ to Rust.

System A shows the low end is an architectural choice, not a property of C: HTTP to SQL in 3 artifacts, no entity, DTO, mapper, or repository classes, 205 methods returning the response body directly, deployed as 8 jars totaling 3.3 MB with a frameworkless 46k-line front end; its deployment discipline is the build-once-promote model described, with the same system anonymized the same way, in [18]. Comparable public JPA systems declare 73 to 482 Maven artifacts (counted from their POM files at the recorded commits; one adds 554 npm packages), each a version to track and a vulnerability feed to watch, none of it visible in a line count.

The author's two Java toolkits from 2009 and 2011, `aisgedcom` and `aisconvert` [11, 12] (8,856 code lines, 461 methods between them), carry the same profile in plain Java a decade before agents existed: median method 7 and 9 lines at complexity 2, complexity maxima of 41 and 44, no Java file changed since their 2015 import, and both compile unchanged under javac 21 today (aisgedcom with its legacy source encoding declared), which is what stable semantics means in practice.

Two guards against over-reading the C rows. Size: the same behavior in this author's C costs up to 2.7 times his Java in lexical tokens, and the agent pays that price in full (mincdp: 6,932 o200k against 2,615); compression shows the excess is mostly repetition (the xz ratio falls to 1.73); and the sites records (Section 3) show why exact repetition is safe: one replace-all pass erases identical copies, and the untested hazard is the near-miss form no single pass ends. The C advantage claimed here is closure locality, never size. Runtime: nothing here rests on machine cost.

5 Why the optimum moved

Objects are for people: a class is a bundle sized to a human working set, an interface is a promise a human can hold whole, and an agent reads in neither unit. Structs are enough for the data and functions over them are enough for the algorithm, which is how both measured plain systems are written, down to the persistence layer that returns rows, not objects, and most visibly in streaming and the hard algorithmics: the ciphers, the compressors, the codecs are functions over buffers in every codebase, whatever the language around them. Abstraction layers compensated for specific human limits, and building them was the correct engineering for the readers of that time; the claim here is that the optimum moved when the reader changed, not that the layers were a mistake when they were built. The functions are separable, and they lose their value separately:

1. **Reliable mechanical edits** (DRY, central change points). The sites experiment did not elicit the failure at 30 exact-copy sites; the near-miss, cross-file form remains open. The indirection cost of every read remains in either case.
2. **Compression for small working memory.** Abstraction is a lossy compressor discovered for the human reader [17]; an agent reads the whole deciding text where it fits beside the work, and retrieves slices where it does not. The marginal value of hiding detail reaches zero where the whole truth fits; whether anything changes sharply past that point is open; what to measure there is the working set after retrieval, not the window boundary, and the experiments are named in Section 6.
3. **Coordination contracts between teams.** Partially survives: a module seam can carry the contract a framework boundary carried; how far team sizes actually fall is not measured here.
4. **Reuse of hard, stable code.** Survives fully: the platform floor. The measured plain systems keep abstraction at floors and drop it as convention.

Corollary, argued rather than measured: languages with stable semantics (C [1], FORTRAN, Ada, plain Java [5, 6]) become viable again, because everything ever written about them describes the same language, one coherent body in the training corpus, while a framework's corpus is a superposition of incompatible versions. The old languages were expensive for unaided humans, and the expense was exactly the bookkeeping agents now do. Corpus coherence has no metric in this paper yet; defining one (how widely the public text about a construct disagrees across versions) is on the roadmap, and until then this paragraph is argument. Precedent for the fast death of a compensating abstraction: calculators ended log tables and slide rules within a decade.

The direction of construction inverts with it: the layers accumulated because building went bottom-up, each generation standing on the last. With an agent, programming runs top-down: state the objective, state the guards, and add one bias to the prompt ("plain C", "plain Java with JDBC, streaming") and the middle between goal and floor stays thin; the closures of Table 2 are the result of that one-line bias. The bias is necessary: an agent's default is its training-set average, the industry idiom, and left undirected it regresses to that mean. Several of the C projects in the author's corpus, `mincdp` among them [10], were rewritten with an AI coder under the same explicit-style direction: the low-closure style is the agent's own output once the bias is stated, and the guidance is what steers it away from the average it learned.

The human work that remains is the objective function (requirements precise enough to implement) and the guards (what the tests must assert). Language choice becomes a measurement: pick what minimizes tokens-to-trace and corpus incoherence, because nobody is paying the human learning cost anymore. Relational access follows the same rule: SQL written against the JDBC contract [6, 7] is one stable text the database runs as written, which is the plain stacks' zero in categories 2 and 3.

6 The experiment, run

Six constructs, each a pair: a small C program and the same behavior in C++ through one construct (a virtual registry, a template, operator overloads, RAII, inheritance, and a mix of them), plus a control pair added at review, the same registry classes in a single file. Four tasks per pair with the same text in both languages, five repetitions, 280 runs, graded by hidden tests the agent never saw; predictions were written 2026-08-24, before any pair existed. The C++ sides follow the anchor era's idiom (classes, a registry, `unique_ptr`), not post-C++17 style. The pairs, the grader, and the raw runs of both executions are beside this paper.

278 of 280 runs passed (the two failures are one C++ cell missing the same empty-store edge twice), so correctness does not distinguish the sides at this size. Turn counts do: the C++ side cost more in all seven pairs, six of six independent constructs, $p = 0.031$, medians 8 against 9, while the money difference, \$0.105 against \$0.109 a run, does not resolve at the construct unit.

Construct	C++ files	C turns	C++ turns	C++ − C
virtual registry, one class per file	8	7.85	11.35	+3.50
mixed constructs	4	7.85	10.65	+2.80
template in a header	2	6.90	8.60	+1.70
RAII	2	8.15	9.20	+1.05
inheritance	2	9.15	9.90	+0.75
virtual registry, single file (control)	1	6.80	7.30	+0.50
operator overloads	2	8.40	8.85	+0.45

Table 3: Mean turns to termination, 20 runs per construct and language; all runs but two ended passing; the two failures sit in the single-file registry's C++ cell and are included in its mean. The C side of both registry rows is the same program. This execution cost \$29.96 and 117 machine minutes.

Three numbers in the table stand above noise: the registry in eight files (+3.50, $p = 0.003$), the mix (+2.80, $p < 0.001$), the template (+1.70, $p = 0.004$). The noise itself is measured: the same C program, run 20 times twice, differs by 1.05 turns. The control row answers the section's question: the same registry in one file costs +0.50, indistinguishable from zero, so the agent pays for the walk across files, not for the virtual dispatch (the cut from 11.35 to 7.30 turns has $p \approx 3 \times 10^{-4}$). We ran the experiment twice; three C++ sides were repaired between the executions, and on the unchanged pairs the small numbers moved (RAII +0.50 then +1.05), so we trust the sign, the three large effects, and nothing finer. One bias ran toward the null and survived: the scattered idiom's file names hint where to look.

The pairs also give the behavior-matched closure comparison the four systems cannot: over the seven pairs the C sides total 3,898 o200k tokens of deciding text against 4,547 for the C++ (source files only, build files excluded), a factor of 1.17 with the single-file control included, 1.13 over the six idiomatic pairs, and in two pairs the C++ side is the smaller (template 0.82, inheritance 0.96). Held to identical behavior, the idiom costs 13 to 17% more text; the 2.8 times between the whole tools is therefore mostly feature scope and what a style accretes at system size, not the constructs.

Of the recorded predictions, only the cost direction held: the prediction said C++ would be cheaper on local tasks, and it was not; the predicted failures never came. A caution on scale: every run sat far below the context window, and an agent retrieves slices rather than loading closures, so what a large closure costs is untested here. The two experiments that would test it are cross-module tasks on taskwarrior itself against its plain twin, and one endpoint of the System A kind built three ways, judged at the HTTP

boundary, with the criterion recorded: the hypothesis fails if the framework tasks do not cost more than the plain ones.

7 Related measurements

The nearest prior work measures token consumption during runs. A study of agent trajectories across Java, OCaml, Python and Rust on LiveCodeBench problems [19] measures consumed tokens per language, difficulty controlled, and finds that the language with the longest programs is not the one that costs the most, which agrees with our pairs result. An analysis of token consumption on SWE-bench Verified [20] finds JVM projects dearer than Python largely through verbose build and dependency logs, an ecosystem cost adjacent to our category 3. Cross-language benchmarks such as MultiPL-E [21] compare pass rates, not cost; SWE-bench [22] prices agents on real repositories with the language mostly fixed; and the token-cost question was asked for natural languages through tokenizers [23]. None of these measures the deciding text of a real endpoint before any run, or classifies it by residence, which is what tokens-to-trace adds. The two kinds of count are complementary, and the static one exists for a reason the spend cannot serve: consumed tokens are a property of the model, the harness and the date, and in our own pilot, whose fixtures are small, they were mostly harness, the source never above 1.1% of a context, though at repository scale the reading itself dominates the spend; the deciding text is a property of the code alone, computable before anything is built. The difference is the context a path demands: for the plain C path, six files the agent can read whole; for the same logic on Spring, the project's part, the framework's interpreters, and conventions no context can hold. A spend count cannot see that difference, because an agent answering from its weights makes the missing context look free.

8 Threats to validity

The C++ we measured is the idiom as it is practiced [3], not the language itself; C-style code compiled as C++ would get the C row's closure. The public subjects show the idiom's average, while both plain systems were written to a stated style that deliberately leaves that average, so the comparison is between what teams write by default and what one line of direction produces. Anyone can state the same line.

One rater derived the closures, and he is the author of ais and one of System A's developers. Each system contributes one endpoint, and he chose which. To check him, a second rater re-derived two closures from the definition alone, without seeing the manifests. On the C++ file list the two agree on 97 of 103 files: the second rater added the status column and the JSON module, and dropped the dependency graph and one implementation file. On the framework path the second rater found 50 classes where the paper counts 27, with 25 of the 27 among them, and his 50 measure 373k tokens against the printed 124k. The printed category 2 is therefore the conservative trace, and its being an undercount is confirmed. taskwarrior is a single registry-heavy codebase, pinned at 2.6.2 because that is the last all-C++ release; leveldb would have measured smaller. On the plain side the own-code objection is bounded by the third-party row: Monocypher's encrypt path, written by another author, closes at the same few thousand tokens as the author's two systems.

Where the platform floor sits is a judgment, so both ends are printed: moving it changes ais by 2.5 times (5.8k to 14.4k) and Spring's category 2 by 2.4 times (124k to 51k). A stricter floor would also turn the plain zeros into small numbers, because `sql_mode`, `getopt` and the build flags were excluded everywhere, and that omission is largest for C. SQL itself is an interpreted notation like the derived-query grammar; it is excused here only because its meaning has been stable for decades, and stability is the property the taxonomy wants; residence is its measurable proxy. Category 2 counts whole classes from one trace that no coverage run has verified, so it bounds in neither direction, and the C++ row is whole files, with the same-granularity comparator printed beside it.

The measure prices reading and nothing else. It does not price the knowledge a reader brings from outside: the years a person spends becoming proficient in C++ or in a framework, and the training corpus

an agent holds for the same purpose, sit on top of every number here. The size of that implicit context can itself be compared by the defining books, K&R's 272 pages against Stroustrup's 1,368 [1, 3], and a framework's reference has no fixed size at all: it moves with the version. It does not price security, which is its own dimension: framework defaults replace hand-rolled code whose defect classes are real, and System A answers only part of that by keeping its secrets in one place, outside the repository and the artifact. It cannot credit what a type system removes: `const` and RAII settle questions the reader of C must verify by hand, so the C++ closures are overcounted and the manual bookkeeping in the ais engine is undercounted. And in certification work under DO-178C or ISO 26262, every duplicated site still has to be verified separately, whatever the agent's edit coverage.

Both agent experiments of Section 3 sat far below the context window and passed everything, so they bound mechanisms and cannot separate representations. In the pairs the C++ file count changes together with the construct, one class per file being the registry idiom, so the top of Table 3 prices construct and layout together. Spring and the RealWorld application are in every training corpus, which will confound the planned endpoint experiment. One model ran, on one date; agent cost is a property of the model and the representation together.

9 Conclusion and recommendation

It is shown, in counts anyone can recompute, that writing in structs and functions, in an old language with stable meaning, is far more compact in the agent's memory: the whole deciding text of a path costs a few thousand tokens and leaves the window to the work, while the layered styles spend the window itself (257k) or keep the deciding text where no repository holds it. The experiments put small numbers on the syntax: held to identical behavior, the C++ sides carry only 1.17 times the C sides' text; below the window a construct of indirection kept in one or two files costs the agent under two turns, differences under a turn sit inside run-to-run noise, almost nothing breaks, and the one large cost, measured and controlled, is the file scatter the idiom brings. What a closure that outgrows the window costs is still open, and its scope is stated: an agent retrieves slices rather than loading wholes, so the question at scale is the working set after retrieval and the text no retrieval reaches, and the two experiments that decide it are named above.

The recommendation, scoped to read cost and to teams working with agents: state the bias in the prompt ("plain C", "plain Java with JDBC, streaming"), keep abstraction only where a platform floor earns it, spend the human effort on the objective (the requirements) and the guards (what the tests must assert; the working practices this compresses are published as recipes [16]), and when choosing a stack, measure the closure of one representative path before arguing about taste.

Method note

Closures measured 2026-08-25: System A private tree; `ais` at head [9]; taskwarrior 2.6.2; the Spring application at v2.1.1 with framework source from the published sources jars of the BOM-resolved versions; every manifest records its version or commit, the public subjects are re-fetchable from the cited repositories, and the exact checkouts are available from the author on request. Tokens are o200k via tiktoken; manifests with per-row lines, characters, and tokens are `closures.py` / `closures.json`. Corpus, duplication, JPA, pilot and site-coverage numbers come from the scripts and JSON beside this paper (`measure.py`, `dup.py`, `external.py`, `jpa.py`, `pilot/`, `sites/`), each recording its commits and raw runs; the blind second rater's derivation is `second_rater.txt`; the Maven-artifact counts are from the projects' POMs and not yet scripted. Region boundaries are function, block, or DDL-statement granularity; whole files where stated.

References

[1] B. Kernighan, D. Ritchie. *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[2] A. Robbins. *Linux Programming by Example: The Fundamentals*. Prentice Hall, 2004.

[3] B. Stroustrup. *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.

[4] MISRA. *MISRA C:2012 Guidelines for the use of the C language in critical systems*. HORIBA MIRA, 2013.

[5] J. Gosling, B. Joy, G. Steele, G. Bracha. *The Java Language Specification*, 2nd ed. Addison-Wesley, 2000.

[6] S. White, M. Hapner. *JDBC 2.1 API Specification*. Sun Microsystems, 1999.

[7] H. F. Korth, A. Silberschatz. *Database System Concepts*. McGraw-Hill, 1986.

[8] *AIS: Coding Style and Ideology*. In [9], `doc/dev/STYLE.md`.

[9] `ais`, an associative index in C99. `https://github.com/Anode1/ais`. The five-language bench and its `LANG_COMPARISON` note are under `tests/perf`.

[10] `mincdp`, a Chrome DevTools Protocol client, one file per language. `https://github.com/Anode1/mincdp`.

[11] `aisgedcom`, a GEDCOM genealogy toolkit in Java, 2009–2011. `https://github.com/Anode1/aisgedcom`.

[12] `aisconvert`, a personal genomics toolkit in Java, 2009. `https://github.com/Anode1/aisconvert`.

[13] L. Vaillant. *Monocypher* 4.0.3, a cryptographic library in C99. `https://monocypher.org`.

[14] `taskwarrior`, an open-source command-line task manager, in development since 2006; 2.6.2 is the last all-C++ release. `https://github.com/GothenburgBitFactory/taskwarrior`.

[15] `realworld-springboot-java` v2.1.1, a Spring Boot implementation of the RealWorld specification (one Medium-style application, implemented identically across many stacks). `https://github.com/raeperd/realworld-springboot-java`; the specification: `https://github.com/gothinkster/realworld`.

[16] `agent-recipes`: working practices for coding with AI agents, generalized from the systems measured here. `https://github.com/Anode1/agent-recipes`.

[17] V. Gavrilov. *Intelligence Is the Discovery of Compressors*. Zenodo, 2026. `https://doi.org/10.5281/zenodo.20440110`.

[18] V. Gavrilov. *The Artifact Promotion Control Model: An Implementation Case Study*. Zenodo, 2026. `https://doi.org/10.5281/zenodo.20528903`.

[19] *The Best Programming Language for Tokenmaxxing: An Investigation of Coding Agent Behavior Across Programming Languages*. arXiv:2607.22807, 2026.

[20] *How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks*. arXiv:2604.22750, 2026.

[21] F. Cassano et al. *MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation*. IEEE TSE, 2023.

[22] C. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.

[23] A. Petrov et al. *Language Model Tokenizers Introduce Unfairness Between Languages*. NeurIPS, 2023.

[24] tiktoken, OpenAI's open-source tokenizer library; `o200k_base` is the encoding of OpenAI's current models. `https://github.com/openai/tiktoken`.